Additional Things I Included Beyond the Assignment

I spent about a week on these project I tried to apply some ideas here which were not asked in the assigment but I felt genuinely useful, not just to add more features I want to walk through each one and explain where the idea came from and why I thought it made sense for this project

Summary

Feature and Why I Added it

1. FastAPI REST service ->   makes the agent usable by other services

2 . AgentResponse dataclass -> needed for debugging and eval

3. Two-layer write protection -> defense-in-depth for the dataset

4. Prompt injection detection ->   basic security for a public endpoint

5. Conversation-style Cypher correction -> more effective than regenerating from scratch

6. Tenacity retry ->   handles transient OpenAI failures

7. Thread timeout   -> prevents hung calls from blocking threads

8. Frozen Settings dataclass -> one place for all config values

9._synthesize_empty()   ->   turns zero results into useful information

10. Flavor profile map ->   domain knowledge the CSVs don't have

11. cuisines[] on PAIRS_WITH ->   makes cuisine-specific pairing queries fast

12. Distinctiveness ratio -> surfaces genuinely characteristic ingredients

13. 9 Cypher examples in prompt -> gives the model concrete templates to follow

14. result_assertion on raw rows -> accuracy that can't be gamed by good writing

15. graceful_handling for edge cases -> right metric for the right kind of question

16 .66 unit tests -> makes the submission verifiable without setup

17.Production docker-compose | stable, memory-tuned environment

18. Auto LIMIT injection | prevents accidental large context window calls

1. FastAPI REST Service (src/api.py)

NEED: The assignment showed a basic Python class with a `query()` method. That works perfectly for demos, but something I kept reading about in ML deployment write-ups is that a class you have to `import` is only half the work. In real team member frontend, mobile another backend service needs to call this over HTTP.

So I wrapped the agent in a FastAPI service with two endpoints:

- `GET /health`- it checks if Neo4j is reachable, returns `ok` or `degraded`

- `POST /query` - takes a question, returns the insight

````
So I wrapped the agent in a FastAPI service with
two endpoints:
- `GET /health`-  it checks if Neo4j is reachable,
returns `ok` or `degraded`
- `POST /query` - takes a question, returns the
insight

```python
class MenuInsightsAgent:
    def query(self, user_question: str) -> str:
        return "some answer"
# vs
# POST /query → { "question": "..." } → {
"insight": "...", "cypher": "..." }
```
````

```

**DECISION:** I hope this shows the agent as something that could actually slot into a real workflow rather than just a standalone script
```

2. AgentResponse Dataclass (7 Fields)

NEED: The assignment asked for the agent to return a string with the answer I noticed that when an agent gives a wrong answer and you only have the output string there's no way to know what went wrong did it query the wrong data? did the synthesis go wrong?

I added a structured return type instead:

````
I added a structured return type instead:

```python
@dataclass
class AgentResponse:
    question:        str    # what was asked
    cypher:          str    # the database query that ran
    raw_results:     list   # actual rows from neo4j
    insight:         str    # the answer the user sees
    latency_seconds: float  # how long it took
    retries:         int    # how many times cypher had to be
    corrected
    request_id:      str    # unique id for tracing
```
````

DECISION: This also turned out to be necessary for the eval - if `query()` just returned a string, I couldn't have written the ground-truth assertions that check the raw database rows

3. Two-Layer Write Protection

NEED: The assignment mentioned handling malformed queries While reading about database security for AI systems I came across the pattern of having two independent checks rather than one specifically because any single check can have edge cases

````
Layer 1 — before the query reaches Neo4j:**
```python
_WRITE_OPS = re.compile(
    r"\b(CREATE|MERGE|DELETE|SET|REMOVE|DROP|DETACH)\b", re.
    IGNORECASE
)
```
If the generated Cypher contains any write keyword, it's rejected
immediately

Layer 2 — at the database level:
```python
with self._driver.session(default_access_mode=READ_ACCESS) as
session:
```
Even if Layer 1 somehow missed something, Neo4j itself refuses to
run writes in a `READ_ACCESS` session.
````

DECISON: The reasoning I found convincing: if both checks have to fail at the same time for a write to go through, the probability of an accident becomes very small Given this dataset has 50,000 rows that would be hard to reload, I felt this was worth including

4. Prompt Injection Detection

NEED:it's apparently one of the most common ways public-facing LLM systems get misuse The pattern is someone sends something like "ignore previous instructions and do X instead"

````
```python
_INJECTION_RE = re.compile(
    r"\b(ignore\s+(all\s+)?(previous|prior|above|instructions)|
    forget\s+(all\s+)?(previous|your)|you\s+are\s+now|jailbreak)
    \b",
    re.IGNORECASE
)
```
````

DECISION: If this pattern is detected in the user's question, the request is blocked before it ever reaches GPT-4o and returns a `400` error. It's a simple regex, not a perfect solution but it catches the most common patterns without adding latency to normal requests

5. Conversation-Style Cypher Correction

NEED :The assignment said to handle errors for malformed queries The basic approach most examples show is if Cypher fails, generate a new one from scratch but what i have done is show the model its own failed query alongside the exact error message, so it can do targeted correction rather than starting over

```
targeted correction rather than starting over

```
Basic we can do these:
cypher fails → ask LLM to generate a new cypher

What I implemented:
cypher fails → show LLM: "you wrote [query], it failed with
[error], please fix it"
```
```

DECISION: For example if the model used `r.restaurant_type` but the property is actually `r.type`, and Neo4j says "Unknown property 'restaurant_type'", the model can pinpoint exactly what to fix. Regenerating from scratch risks making the same mistake again

6. Tenacity Retry for Transient LLM Errors

NEED :OpenAI has occasional rate limit errors and timeouts, especially under load. I came across the `tenacity` library in a few reliability engineering posts and it seemed like a clean way to handle this without writing manual retry loops.

```python
_LLM_RETRY = retry(
    retry=retry_if_exception_type((RateLimitError,
    APITimeoutError, APIConnectionError)),
    wait=wait_exponential(min=2, max=30),
    stop=stop_after_attempt(3),
)
```

DECISION: It waits 2 seconds, then 4, then 8 before giving up — exponential backoff. The user doesn't see anything different, the request just takes a few extra seconds. Without this, a temporary OpenAI hiccup would surface as an error to the user even though retrying once would have worked

7. ThreadPoolExecutor Timeout (45s Hard Cap)

NEED :This came from reading about production incident write-ups where a single hanging LLM call blocked a thread indefinitely. The pattern I found was to run the blocking call in a thread pool and set a timeout on `future.result()`.

```
I found was to run the blocking call in a thread pool and set a
timeout on `future.result()`.

```python
with concurrent.futures.ThreadPoolExecutor(max_workers=1) as
executor:
    future = executor.submit(self._query_impl, question, rid)
    response = future.result(timeout=45)
```
```

DECISION: If the OpenAI call hangs for any reason, after 45 seconds it's cancelled and the caller gets a clear `504 timeout` error instead of waiting forever. It's a small thing but it felt important for a service that might handle multiple concurrent requests

8. Frozen Dataclass for Settings (config.py)

````
```python
@dataclass(frozen=True)
class Settings:
    neo4j_uri:               str   = "bolt://localhost:7687"
    cypher_model:            str   = "gpt-4o"
    cypher_temperature:      float = 0.0
    synthesis_temperature:   float = 0.4
    max_retries:             int   = 2
    query_timeout_seconds:   int   = 45
```
````

NEED :Having one file for all configuration including things that aren't in env vars like temperatures and retry counts — made the code easier to follow while I was working on it. `frozen=True` prevents anything from accidentally overwriting a setting during a run

9.synthesize_empty() for Zero-Result Responses

NEED :The assignment asked the agent to synthesize insights from results. I thought about what happens when the database returns nothing the first response is just "no results found," which i thought isn't very useful

```
```python
def _synthesize_empty(self, question: str, cypher: str) -> str:
    # tells the LLM: the query returned zero rows.
    # frame this as what the absence means, not just that nothing
    was found.
```
```

DECISION:

So absence of data is itself information. If someone asks "what kimchi-chocolate pairings exist in NYC restaurants" and the answer is none, that's actually a potential white-space opportunity not a dead end So instead of "no results," the agent says something like "this combination doesn't appear in the dataset which could mean no NYC restaurant has tried it yet"

10. Hand-Coded Flavor Profile Map (100+ Ingredients)

NEED :The assignment mentioned "consider adding flavor profiles" as a suggestion without much detail. The source CSVs have no flavor information at all, so I spent some time manually building a map of 100+ ingredients to 7 flavor profiles

```python
FLAVOR_PROFILE_MAP = {
    "spicy":  ["chili powder", "sriracha", "jalapeño",
    "cayenne", ...],
    "umami":  ["soy sauce", "miso", "mushroom", "parmesan", ...],
    "sweet":  ["sugar", "honey", "maple syrup", "mango", ...],
    "sour":   ["lemon", "lime", "vinegar", "yuzu",
    "tamarind", ...],
    "smoky":  ["bacon", "chipotle", "smoked paprika", ...],
    "herbal": ["basil", "cilantro", "thyme", "lemongrass", ...],
    "savory": ["beef", "chicken", "garlic", "butter", ...],
}
```

DECISION: Without this the agent can't meaningfully answer questions like "what flavors are trending in NYC desserts" because the graph has no concept of what "sweet" or "smoky" means. This required thinking about food more than programming, which I actually found kind of interesting to do

11. cuisines[] Array on PAIRS_WITH Edges

NEED :The assignment specified ` PAIRS_WITH {frequency: N}` just the count of how often two ingredients appear together While planning the graph schema I realised that if someone asks "which ingredient pairings are specific to Japanese cuisine," you'd have to recount co-occurrences at query time, which would be slow.

````
```cypher
MERGE (i1)-[r:PAIRS_WITH]-(i2)
SET r.frequency = freq,
    r.cuisines  = ['Japanese', 'Korean', 'American']
```

Storing the cuisines list on the edge at load time means
cuisine-specific pairing queries become a simple filter:

```cypher
WHERE 'Japanese' IN r.cuisines
```

It's a small schema addition but it makes a class of useful
questions much faster to answer
````

DECISION: It's a small schema addition but it makes a class of useful questions much faster to answer

12. Distinctiveness Ratio Pattern in the System Prompt

NEED :This one came from thinking about what useful means for a cuisine analysis question. If someone asks "what spices are used in Indian cuisin" ordering by raw count gives you garlic and salt at the top they appear everywhere, not just Indian food

I came across the idea of a distinctiveness ratio in a data analysis context and thought it applied well here:

````
```
pct_in_cuisine = (appearances in Indian menus) / (total
appearances) × 100
```
````

DECISION: Turmeric appears in Indian menus ~80% of the time it appears anywhere Garlic appears in Indian menus only ~15% of the time. So turmeric ranks much higher, which is the actually useful answer.

I included a worked example of this pattern in the system prompt so the agent applies it automatically when the question calls for it

13. Nine Annotated Cypher Examples in the System Prompt

NEED :The approach I read about most consistently for improving LLM-to-query translation was few-shot prompting giving the model real, working examples rather than just describing what you want. I wrote 9 examples covering different query patterns:

```
- basic ingredient frequency lookup
- PAIRS_WITH without duplicate pairs (`WHERE id(i1) < id(i2)`)
- cross-cuisine comparison with the ratio approach
- white-space discovery (common in one cuisine, rare in another)
- neighborhood-filtered queries
- price aggregation
- chain vs independent comparison
```

DECISION: Without examples the model has to guess at the schema and Cypher syntax With 9 concrete templates it has enough coverage to handle most question types by analogy. I think this contributed a lot to why accuracy came out at 1.00 across all 20 eval cases

14. Ground-Truth result_assertion Callables in the Eval

NEED :The assignment asked for scoring functions including one for accuracy. The standard approach is to use an LLM judge — ask the model to score how accurate the answer seems. I was a bit skeptical of this for accuracy specifically because a model can write a confident, well-worded answer that's based on wrong data and an LLM judge might score it highly.

I came across the idea of testing raw database rows (before the synthesis step).

```python
{
    "input": "Most common ingredients in ramen?",
    "result_assertion": lambda rows: len(rows) >= 5 and
    _has_string_and_num(rows)
}
```

DECISION: The callable runs against `response.raw_results` — the actual Neo4j rows, before the LLM wrote anything. If the data is wrong, the assertion fails regardless of how well the insight is phrased This felt like a more honest measure of accuracy

15. graceful_handling Replacing creativity for Edge Cases

NEED :The assignment specified four scoring dimensions including creativity. While writing the eval test cases I noticed that the three edge cases (vague question, sparse results, typo in the question) aren't really questions where you'd want to score creativity they're questions where the right behavior is to handle the awkward situation gracefully.

Scoring "Find vegan options in steakhouses" on creativity when the database returns almost nothing didn't seem meaningful. What matters is whether the agent communicated the scarcity honestly and helpfully.

So I wrote a separate scorer `score_graceful_handling` and swap it in only for edge case questions. It asks the LLM judge: did the agent handle ambiguity, sparse data, or typos in a way that was still useful to the user?

16. 66 Unit Tests Across 4 Files

NEED :The assignment didn't mention testing at all. I included it because something I read made me think about how a reviewer would actually trust the code if I submit something and claim it works, tests are the evidence rather than just asking someone to take my word for it.

```
- `test_agent.py` — tests for `_clean_cypher` (strips code fences,
rejects write ops, handles edge cases), input validation, and a
mocked query flow
- `test_api.py` — HTTP-level tests (does `/health` return 200?
does a timeout return 504? does a too-long question get rejected?)
- `test_database.py` — tests for the Neo4j wrapper, including the
auto-LIMIT behavior
- `test_evaluate.py` — tests for the scoring functions themselves
```

DECISION: Everything is mocked so no database or OpenAI key is needed to run them They complete in under 7 seconds on any machine I hope this makes it easier for someone reviewing the project to verify things quickly without having to set up the full environment first

17. Production-Oriented Docker Compose

NEED :The assignment showed a basic `docker run` command for Neo4j. I adjusted this to use a docker-compose file with a few settings I came across as important for graph databases under load:

```yaml
image: neo4j:5.20          # pinned version rather than "latest"
NEO4J_PLUGINS: '["apoc"]'   # APOC library enabled
heap_initial: 512m
heap_max: 2G               # memory tuned for 151k relationships
volumes:
  - neo4j_data:/data       # data persists across container restarts
```

Using `neo4j:latest` means the behavior could change on any future `docker pull`. Pinning to `5.20` keeps the environment consistent. The memory settings matter because the default Neo4j heap is quite small and can cause slow queries or out-of-memory errors once the dataset is loaded.

18. Automatic LIMIT Injection in database.py

NEED :This one I added after noticing that LLMs sometimes forget to include a `LIMIT` clause in generated Cypher. A query like `MATCH (m:MenuItem) RETURN m` without a limit would try to return all 50,000 rows and push them into the GPT-4o context window — expensive, slow, and likely to fail.

````
This one I added after noticing that LLMs sometimes forget to
include a `LIMIT` clause in generated Cypher. A query like `MATCH
(m:MenuItem) RETURN m` without a limit would try to return all 50,
000 rows and push them into the GPT-4o context window — expensive,
slow, and likely to fail.

```python
if "RETURN" in cypher.upper() and "LIMIT" not in cypher.upper():
    cypher = cypher.rstrip(";") + f"\nLIMIT {limit}"
```
````

The database wrapper silently adds a `LIMIT 100` if the generated query doesn't already have one. Neither the user nor the LLM notices it — it's just a quiet safety net at the infrastructure level that prevents an accidental expensive call.